Programa:	CloneDVD 3.6
Protección:	ASProtect + Trial 14 días
Objetivos:	Desempaquetar y eliminar el trial
URL descarga:	www.dvd-x-player.com
Dificultad:	Muy fácil
Herramientas:	OllyDbg 1.10 + PeiD + Import Reconstructor
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos] Fecha: 06-09-2005

Hola a todos una vez más..!!

Ante todo dar un saludo a todo aquél que esté leyendo esto y tenga ganas de aprender, en eso estamos todos, es nuestra lucha diaria. En cuanto al cracking hoy nos las veremos con otro ASProtect en una de sus versiones más modernas, aunque he de reconocer que no tiene mucha diferencia respecto a otros más normalitos, ni siquiera se parece a los 1.2x-1.3x que todos hemos visto con esos calls espantosos tipo antidump que [ZekkT] resolvió estupendamente, un saludo para él y para todos vosotros, vamos allá...

Desempakando un ASProtect más...

Este es el mejor título que encontré ya que hoy día ¿quién no ha descomprimido algún que otro ASProtect a mano? Recordemos que no hará mucho era toda una odisea desde nuestro amigo SoftIce y sólo muy muy pocos eran capaces de hacerlo (desde luego yo no me incluyo hice uno y de milagro, jeje..) aquellos días en que AuspeX, Ricardo Narvaja, DGRojo, +tesph, Arcanian y compañía nos asombraban a todos con sus tutoriales, afortunadamente éstos sirvieron para que ahora todos nosotros hayamos aprendido a hacerlo por nosotros mismos, incluso hay quien se atreve con otros más potentes, mi admiración para todos ellos por supuesto. Pero esto como digo quedó en el pasado, a excepción del nuevo Asprotect2 el resto de versiones se han convertido en una tarea casi mecánica que cualquier cracker que haya leído un mínimo puede vencer. Dicho esto pasemos al atake. Diré que este tutorial no diferirá mucho de los otros ASProtect que vimos, pero el que ya haya buenos textos no es motivo suficiente para que se deje de escribir sobre una protección.

Abrimos el exe con PEiD o RDG Packer y nos dice lo siguiente:

ASProtect 1.2x - 1.3x [Registered] -> Alexey Solodovnikov

Vale, ahora ya sabemos quien es el otro, je.. cargamos el programa en Ollydbg 1.10 con el plugin IsDebuggerPresent activado para no ser detectado, este packer no tiene ningún método de detección además de este, ocultando el Olly carga sin problemas. Nuestro primer objetivo será encontrar la última excepción, el método está más que expllicado, dar Run, superar con Shift+F9 las excepciones hasta que cargue el programa y anotar cual fue la última en que olly paró, en este caso fue en esta:

Address	Hex dump	Disassembly	Comment
0219FB34	C601 4E	MOV BYTE PTR DS:[ECX],4E	
0219FB37	A4	MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]	
0219FB38	D5 94	AAD 94	
0219FB3A	9F	LAHF	

Última excepción: 219FDC3 / 219FB34

La anotamos como siempre, también he apuntado otra al lado ya hay veces que cambia de excepción, sobretodo cuando reparamos la tabla con el mágico, anotamos ambos lugares para luego, de momento reiniciamos Olly con Ctrl+F2 y llegamos a la última excepción, antes de pasarla ponemos el BPM en la primera sección como hacemos siempre y luego sí la superamos, de esta manera llegamos al OEP:

Address	Hex dump	Disassembly	Comment
0041EBCA	55	PUSH EBP	
0041EBCB	8BEC	MOV EBP,ESP	
0041EBCD	6A FF	PUSH -1	
0041EBCF	68 A0B64200	PUSH CloneDVD.0042B6A0	
0041EBD4	68 34ED4100	PUSH CloneDVD.0041ED34	JMP to msvcrt._except_handler3
0041EBD9	64:A1 00000000	MOV EAX,DWORD PTR FS:[0]	
0041EBDF	50	PUSH EAX	

OEP: 41EBCA

Vale lo anotamos también, el que quiera puede dumpear ahora pero mejor yo lo hago después (una preferencia personal sin importancia) aprovechando que estamos aquí buscaremos la IAT, y la delimitaremos también, buscando su inicio y final, con ambos datos obtendremos su longitud restándolos después. Nos situamos en un JMP o CALL a la IAT y hacemos un Follow in dump->

Memory Address para caer en plena tabla, a partir de aquí se trata de ir subiendo a ver donde empieza la misma, encontramos su inicio aquí:

```
Address   Hex dump                                          ASCII
00428000  BB 28 DA 77 2F 58 DC 77 CD 2F DA 77 D8 17 DA 77  ╗(┌w/X▄w═/┌w╪↨┌w
00428010  10 24 DA 77 FD 31 DA 77 B1 63 DB 77 9A 22 DA 77  ►$┌w²1┌w▒c█w Ü"┌w
00428020  00 00 00 00 00 00 72 02 00 00 73 02 00 00 74 02  ......r☻..s☻..t☻
00428030  00 00 00 00 10 C8 00 10 20 CA 00 10 00 00 00 00  ....►╚.► ╩.►....
```

inicio iat: 428000

lo anotamos también, en este caso no es difícil encontrarlo porque coincide con el inicio de la sección, hagamos lo propio con el final:

```
Address   Hex dump                                          ASCII
00428930  FA 27 D6 77 00 00 00 00 7D 16 BD 77 2D 16 BD 77  ·'╓w....}▬¢w-▬¢w
00428940  E3 15 BD 77 00 00 00 00 62 8B B0 76 00 00 00 00  ╙§¢w....b╜▒v....
00428950  CF 90 36 76 90 23 36 76 00 00 00 00 EA 77 AF 01  ╧É6vÉ#6v....╩w»☺
00428960  00 00 00 00 00 00 00 00 D0 24 41 00 70 89 42 00  ........╨$A.pëB.
```

final iat: 428960

le damos un pequeño margen de 1 dword para que el Import vea todos los punteros y no deje ninguno fuera, más vale que sobre que no falte, con estos dos valores obtenemos el largo:

largo = final - inicio => 428960 - 428000 = 960

largo iat: 960

ya tenemos los datos referentes a la IAT, pero si vamos al Import ahora nos dirá que hay muchos punteros inválidos, esto es así porque no hemos buscado el mágico todavía, vayamos por él, busquemos un puntero redirigido:

```
Address   Hex dump                                          ASCII
00428000  BB 28 DA 77 2F 58 DC 77 CD 2F DA 77 D8 17 DA 77  ╗(┌w/X▄w═/┌w╪↨┌w
00428010  10 24 DA 77 FD 31 DA 77 B1 63 DB 77 9A 22 DA 77  ►$┌w²1┌w▒c█w Ü"┌w
00428020  00 00 00 00 00 00 72 02 00 00 73 02 00 00 74 02  ......r☻..s☻..t☻
00428030  00 00 00 00 10 C8 00 10 20 CA 00 10 00 00 00 00  ....►╚.► ╩.►....
```

Allí marcamos tres consecutivos, apuntan a zonas del tipo 27xxxxx que cae en un espacio de memoria que prepara el propio ASProtect, por eso en nuestro dumpeado no existirá, precisamente porque le hemos quitado el compresor de encima, de ahí nace la necesidad de buscar el mágico y evitar así la redirección de la IAT. Reiniciamos Olly y ponemos un BPM On Write en uno de los punteros malos, luego pulsamos F9 para dejarlo correr, para que vaya más rápido podemos tildar las excepciones para que no pare en ninguna ahora y centrarnos en la tabla (esto es opcional y sólo por comodidad), pararemos aquí:

```
0218996A    881C3E          MOV BYTE PTR DS:[ESI+EDI],BL
0218996D    FF45 F0         INC DWORD PTR SS:[EBP-10]
02189970    FF4D EC         DEC DWORD PTR SS:[EBP-14]
```

Pero vemos que va escribiendo byte a byte en la entrada pero además no escribe los bytes finales que eran 2720000 en mi máquina, entonces damos F9 y para en este otro lado:

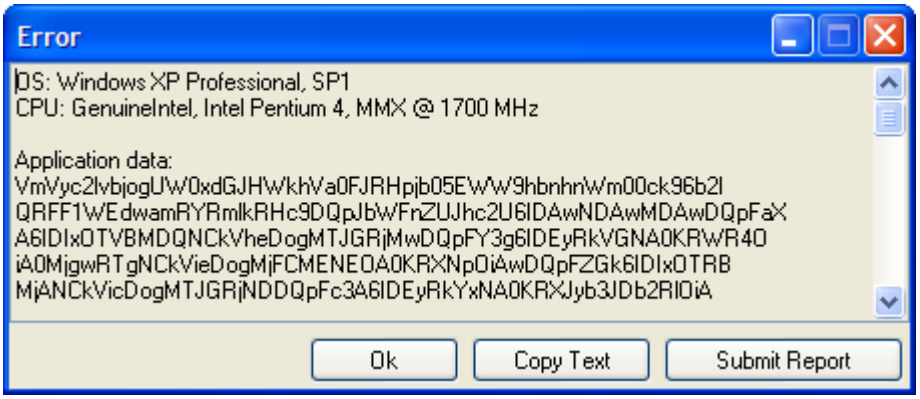
```
Address   Hex dump     Disassembly                          Comment
021958B3    50         PUSH EAX
021958B4    56         PUSH ESI
021958B5    E8 8AFDFFFF CALL 02195644
021958BA    8BD8       MOV EBX,EAX      <- en EAX la API correcta
021958BC    6A 00      PUSH 0
021958BE    68 204A1902 PUSH 2194A20
021958C3    8D4D FC    LEA ECX,DWORD PTR SS:[EBP-4]
021958C6    8BD3       MOV EDX,EBX
021958C8    8BC6       MOV EAX,ESI      redirige las API's
021958CA    E8 FDF8FFFF CALL 021951CC   <-
021958CF    8B55 0C    MOV EDX,DWORD PTR SS:[EBP+C]
021958D2    8B12       MOV EDX,DWORD PTR DS:[EDX]
021958D4    8902       MOV DWORD PTR DS:[EDX],EAX  <- va rellenando la tabla
021958D6  v E9 90000000 JMP 0219596B
```

Según el dibujo llegamos a :21958B5 y al pasar por el CALL nos deja la API correcta en EAX que pasa a EBX en la siguiente línea ya que pierde su valor en :21958C8, no obstante al meterse en el CALL 021951CC pierden todos los registros el valor que tenían, dejando en EAX un valor de los que no nos gustan y pondrá en la IAT un poco después, lo único que vemos es el nombre de la API en EBX a la salida de ese call, no es mucho pero almenos sabemos que api está tratando, no obstante esto se puede repar mediante un script o un injerto, el primer caso suele ser más acorde a esta protección debido al CRC del archivo que detecta los posibles cambios que hagamos, entonces nos queda la opción del injerto. No obstante antes probemos a nopear el call y cambiar EAX por EBX en :21958D4 para que guarde el valor corrento en lugar del malo que pondría en EAX dentro del call que recordemos acabamos de nopear, entonces cambiamos de momento:

```
021958CA    E8 FDF8FFFF     CALL 021951CC                  POR:
021958CA    90 90909090     NOP'S

además de:
021958D4    8902            MOV DWORD PTR DS:[EDX],EAX     POR:
021958D4    891A            MOV DWORD PTR DS:[EDX],EBX           ; comctl32.ImageList_Remove
```

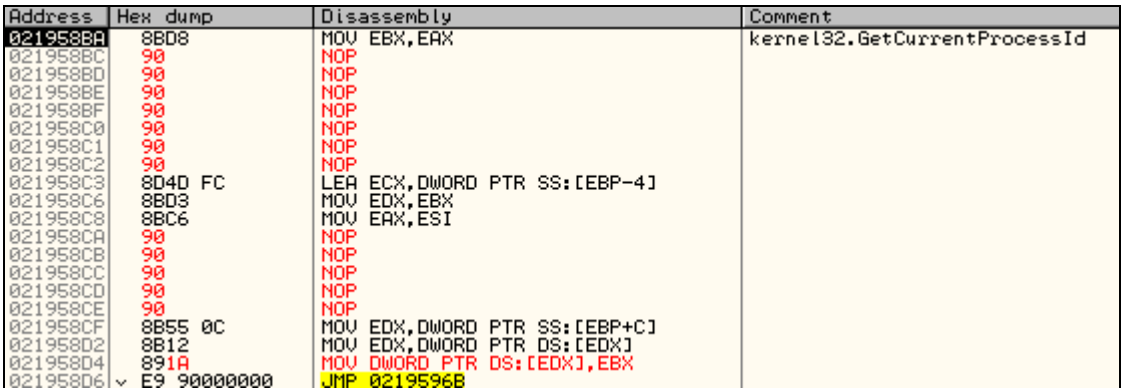
Mirad como sólo apareció el nombre de la API, esto es porque Olly sabe que el valor de EBX corresponde con la dirección de la API que muestra, vale bien parece correcto, damos RUN y...



Esto sucedió..., en un principio pensé que se debía al famoso CRC del packer, aún así no dejé esta línea de ataque y como veremos dará sus frutos, me dio por mirar la modificacion de la pila (registro ESP en 32 bits) que hacía el CALL que acabamos de nopear, para esto tomé el valor de este registro antes y después de la llamada y vi:

Antes ESP==12FDDC
Después ESP==12FDE4
Diferencia +8 bytes

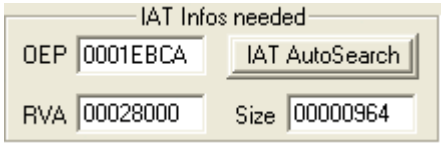
Quiere decir que hay un descuadre de pila de dos dwords, pues eran 8 bytes que son 8*8=64 bits, cada registro son 32 bits, por tanto son 2 registros equivalentes a 2 dwords de desfase, entonces si nopeo el call debo cuadrar ESP, muy probablemente este sea el motivo del error anterior, encima del CALL vemos dos PUSH, justo lo que buscamos, si los nopeamos dejamos de meter en la pila las dos dwords que nos sobran, depués nopeamos también el CALL:



Address	Hex dump	Disassembly	Comment
021958BA	8BD8	MOV EBX,EAX	kernel32.GetCurrentProcessId
021958BC	90	NOP	
021958BD	90	NOP	
021958BE	90	NOP	
021958BF	90	NOP	
021958C0	90	NOP	
021958C1	90	NOP	
021958C2	90	NOP	
021958C3	8D4D FC	LEA ECX,DWORD PTR SS:[EBP-4]	
021958C6	8BD3	MOV EDX,EBX	
021958C8	8BC6	MOV EAX,ESI	
021958CA	90	NOP	
021958CB	90	NOP	
021958CC	90	NOP	
021958CD	90	NOP	
021958CE	90	NOP	
021958CF	8B55 0C	MOV EDX,DWORD PTR SS:[EBP+C]	
021958D2	8B12	MOV EDX,DWORD PTR DS:[EDX]	
021958D4	891A	MOV DWORD PTR DS:[EDX],EBX	
021958D6	E9 90000000	JMP 0219596B	

Ya no se puede quejar, por lo menos por este motivo, la pila queda totalmente ajustada, podemos poner un BP en :021958D6 y ver como se van guardando API's buenas en la IAT y no malas como pretendía ASProtect, antes de que termine de rellanarla toda destildamos las excepciones para que se detenga en ellas, y al llegar a la última recordando que puede variar ponemos el BPM en la primera sección para parar en el OEP, notad que ni siquiera haciendo el Undo Selection llegamos al OEP sin problemas ¿?

Es el turno del Import Reconstructor, abrimos el proceso, le damos los datos:

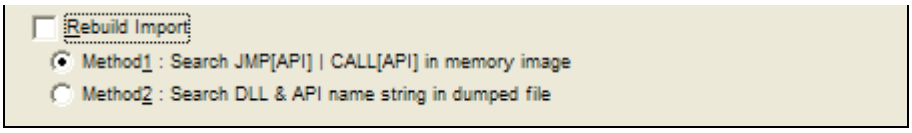


Recordando que en el Import trabajamos en offsets, o sea la dirección real restándole la imagebase que es 400000, esto para los campos OEP y RVA que es el offset del inicio de la IAT, ponemos su largo y damos a Get Imports, nos dice que quedó un puntero por resolver, bien sólo es uno, veámoslo:

IATrva:	Apunta a:	API correcta
28184	21946B8	GetProcAddress

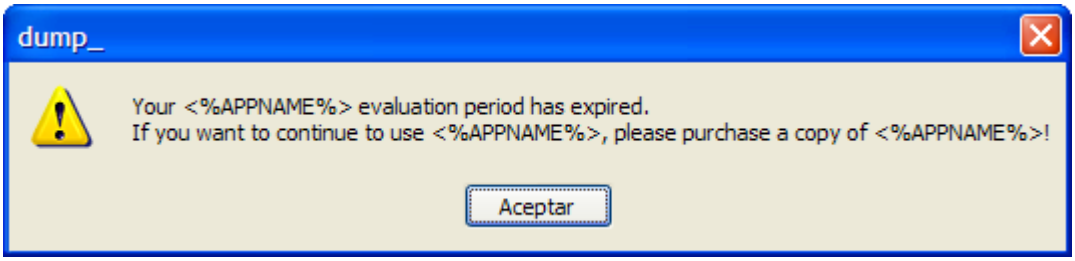
Esta no necesita comentarios, es la más que conocida GetProcAddress presente en todas las tablas que destroza ASProtect, todavía no recuerdo una IAT que no la redirija, es clienta fija, jeje...

La reparamos desde aquí mismo, antes de nada obtendremos nuestro dumpeado:



Esto sobretodo dejarlo como lo véis en la imagen sino Olly intentará reconstruir la IAT por nosotros y no queremos, para eso tememos el Import no? Guardamos el dumpeado como dump.exe por ejemplo, luego retornamos al Import y pulsamos Fix Dump, elegimos el dumpeado que acabamos de crear y nos creará otro archivo al lado llamado dump_.exe este es el que tiene la IAT reparada, que si probamos...

EEEEEEEEEI funciona..!! pero no nos alarmemos, si miramos bien vemos que dice '0 days left!' pero si lo acabamos de instalar, ah ya sé! Un truquito antidump, si lo dumpeamos automáticamente expira para que no lo podamos utlizar, jaja.. que se lo ha creído vaya, ya que hemos llegado hasta aquí seguiremos hasta el final, pulsamos Later y...



vamos que lo dice en serio y todo, carguemos el dumpeado en el Ollydbg y traceemos un poquitín hasta que aparezca la nag y el mensaje, luego busquemos un salto que nos aleje del peligro, esto ocurre aquí:

```
004082C7   .  E8 24290000    CALL dump_.0040ABF0
004082CC   .  85C0           TEST EAX,EAX
004082CE   .  75 53          JNZ SHORT dump_.00408323
004082D0   .  8BCE           MOV ECX,ESI
004082D2   .  E8 A9000000    CALL dump_.00408380                ; Muestra la NAG
004082D7   .  E8 642A0000    CALL dump_.0040AD40
004082DC   .  8BC8           MOV ECX,EAX
004082DE   .  E8 FD250000    CALL dump_.0040A8E0
004082E3   .  85C0           TEST EAX,EAX
004082E5   .  74 3C          JE SHORT dump_.00408323
004082E7   .  8D4C24 04      LEA ECX,DWORD PTR SS:[ESP+4]
004082EB   .  68 94000000    PUSH 94                            ; /Arg2 = 00000094
004082F0   .  51             PUSH ECX                           ; |Arg1
004082F1   .  E8 AA070000    CALL dump_.00408AA0                ; \dump_.00408AA0
```

```
004082F6   .  8B00           MOV EAX,DWORD PTR DS:[EAX]
004082F8   .  6A 00          PUSH 0
004082FA   .  6A 00          PUSH 0
004082FC   .  50             PUSH EAX
004082FD   .  C78424 580100> MOV DWORD PTR SS:[ESP+158],0
00408308   .  E8 23600100    CALL <JMP.&mfc42.#1200>             ; Muestra el MENSAJE
```

En esta parte del código se encarga de mostrar la nagscreen y poco más abajo el mensaje EXPIRED, muy sospechoso que entre ambos haya un salto condicional hacia 408323 que muy casualmente se repite en :4082CE donde hay otro salto a esta dirección que de producirse no vemos ni mensaje ni nag, jaja... reiniciamos Olly, ponemos un BP en :4082CE cuando pare cambiamos el flag zero obligando a que salte...

Pues que va a pasar que carga el programa sin más, el call a mirar es este:

```
004082C7   .  E8 24290000   CALL dump_.0040ABF0
004082CC   .  85C0          TEST EAX,EAX
004082CE   .  75 53         JNZ SHORT dump_.00408323
```

está claro que debe retornar EAX==1 para que salte, esto lo conseguimos si el subcall:

```
0040AC49  |.  E8 82910000   CALL dump_.00413DD0                  ; \dump_.00413DD0
0040AC4E  |.  8D4C24 0C     LEA ECX,DWORD PTR SS:[ESP+C]
0040AC52  |.  8BF0          MOV ESI,EAX
```

retorna EAX==1, que a su vez se consigue si el sub/subcall, jeje..

```
00413E4C  |.  E8 4FAC0000   CALL dump_.0041EAA0
00413E51  |.  0BC2          OR EAX,EDX
00413E53  |.  75 18         JNZ SHORT dump_.00413E6D
```

retorna EAX==0, y modificando la instrucción:

```
00413E41  |. /75 2A         JNZ SHORT dump_crk.00413E6D     POR:
00413E41      90 90         NOP'S
```

Recordad que esto lo hacemos para que el CALL 413DD0 retorne EAX==1.

Y como todo es una cadena, necesitamos cambiar:

```
0041EB0C  |.  F7D8          NEG EAX
0041EB0E  |.  83DA 00       SBB EDX,0
0041EB11  |>  5B            POP EBX
0041EB12  \.  C2 1000       RETN 10
```

POR:

```
0041EB0C      33C0          XOR EAX,EAX
0041EB0E  |.  83DA 00       SBB EDX,0
0041EB11  |>  5B            POP EBX
0041EB12  \.  C2 1000       RETN 10
```

Y cerramos el ciclo, tenemos la cadena de retornos asegurada, probamos el exe modificado y funciona a la perfección, concluímos que tenemos hecho el trabajo ;-)

Vemos que la opción de registro se ha tornado grisácea, esto es porque posiblemente ASProtect se encargaba de esta tarea y al no estar presente no hay queén la haga, sólo queda una referencia a Unregistered en el About.

Mirando un poco el tema descubrí que la opción de registro se apagaba tras sortear el call de la nag y del EXPIRED y que además el que expire el programa no es ningún antidump sinó debido al nombre del archivo, renombrando el dump_.exe a CloneDVD.exe recuperamos el Trial de 14 días nuevamente.

El tema del About es muy sencillo evitar el Unregistered y sin utilizar UltraEdit, como es de dificultad mínima lo dejo en vuestras manos, si alguien no lo consigue o no tiene ganas pues me lo dice y lo comentaré ok?

Para cualquier sugerencia o lo que sea me encontraréis en:

syxe00@yahoo.es
syxe00@hotmail.com